MELODY

Application musicale collaborative

Documentation technique et guide de déploiement

Version 2.1

Symfony 6.4 LTS · PHP 8.2+ · Doctrine ORM · MySQL 8 · Docker

Alex Alkhatib

MIT License

Sommaire

1. Présentation et objectifs · 2. Fonctionnalités · 3. Stack technique · 4. Architecture · 5. Modèle de données · 6. Sécurité · 7. Installation locale · 8. Configuration · 9. Tests · 10. Déploiement Docker · 11. Interface et lecteur · 12. Refonte 2.0 · 13. Roadmap · 14. Licence

1. Présentation et objectifs

Melody est une application web de gestion de **playlists musicales collaboratives**. Les utilisateurs alimentent un catalogue commun de morceaux (titre, artiste, genre, pochette, fichier audio) puis composent leurs propres playlists. Un back-office d'administration permet de gérer le catalogue, les comptes et les messages reçus.

Objectifs du projet

- Concevoir une application Symfony complète et maintenable (MVC en couches).
- Mettre en place une authentification et une gestion de rôles robustes.
- Manipuler Doctrine ORM : relations N-N, ManyToOne, migrations versionnées.
- Livrer une interface responsive, accessible et cohérente.
- Industrialiser : tests automatisés, image Docker, documentation.

2. Fonctionnalités

Domaine	Fonctionnalités
Comptes	Inscription, connexion, « se souvenir de moi », limitation des tentatives, déconnexion sécurisée, profil et changement de mot de passe.
Catalogue	Ajout de morceaux avec pochette et audio, recherche plein texte, pagination, fiche détaillée, lecteur intégré, modification et suppression par le propriétaire.
Playlists	Création, renommage, suppression, sélection multiple de morceaux, compteur automatique, accès restreint au propriétaire.
Contact	Formulaire validé, message persisté puis envoyé par email (expéditeur applicatif + Reply-To visiteur), consultation dans le back-office.
Back-office	Tableau de bord chiffré, CRUD morceaux, genres, playlists et utilisateurs, messages en lecture seule, accès ROLE_ADMIN.

3. Stack technique

Composant	Version	Rôle
PHP	>= 8.2 (Docker : 8.3)	Runtime
Symfony	6.4 LTS	Framework applicatif
Doctrine ORM	2.20	Persistance et mapping
Doctrine Migrations	3.x	Schéma versionné
Twig	3.8	Moteur de templates
EasyAdmin	4.29	Back-office
VichUploaderBundle	2.9	Uploads (pochettes, audio)
KnpPaginatorBundle	6.3	Pagination
Design system	CSS natif + JS vanilla	Interface sombre responsive
MySQL / MariaDB	8.0+	Base de données

Composant	Version	Rôle
PHPUnit	9.6	Tests fonctionnels

4. Architecture

Architecture MVC en couches. La requête traverse le pare-feu de sécurité, puis le contrôleur (validation, autorisations) qui s'appuie sur les formulaires, les entités, les repositories et Twig pour le rendu.

```
Requete HTTP
   |
   +-- public/index.php (front controller)
   |
   +-- Securite : pare-feu 'main' + access_control
   |
   +-- Controller --+-- Form (validation)
   |                +-- Entity (modele + contraintes)
   |                +-- Repository (requetes Doctrine)
   |
   +-- Twig (templates heritant de base.html.twig)
```

Dossier	Contenu
src/Controller	Home, Security, Registration, Morceaux, Playlist, Profile, Contact
src/Controller/Admin	Dashboard et 5 contrôleurs CRUD EasyAdmin
src/Entity	User, Morceaux, Playlist, Genre, Contact
src/Form	6 formulaires typés (inscription, morceau, playlist, contact, profil, mot de passe)
src/Repository	Requêtes : recherche, propriétaire, compteurs
src/Security	LoginAuthenticator (email + mot de passe)
templates	Partials, CRUD, profil, sécurité, admin, pages d'erreur
tests/Functional	Suite de bout en bout sur base SQLite jetable

5. Modèle de données

Table	Colonnes principales	Relations
user	id, email (unique), roles, password, nom, prenom	1-N playlist, 1-N morceaux
genre	id, nom_genre	1-N morceaux
morceaux	id, nom_morceaux, artiste, date_sortie, image_name, music_name, genreid_id, user_id	N-1 genre, N-1 user, N-N playlist
playlist	id, nom_playlist, nb_morceaux, userid_id	N-1 user, N-N morceaux
playlist_morceaux	playlist_id, morceaux_id	Table de jointure (cascade)
contact	id, prenom, email, subject, message, created_at	—

6. Sécurité

Risque	Mesure en place
Accès et privilèges	access_control explicite ; /admin réservé à ROLE_ADMIN ; plus d'édition publique des rôles
Mots de passe	Hachage auto (bcrypt/argon2) et re-hachage automatique
Force brute	login_throttling : 5 tentatives par 15 minutes
CSRF et suppressions	Jetons Symfony partout ; suppressions en POST uniquement
IDOR	isOwnedBy() / isEditableBy() vérifiés côté serveur, puis 403
Uploads, XSS, cookies	Contraintes MIME et taille, dossier media non exécuté, auto-échappement Twig, HttpOnly + SameSite

7. Installation locale

```
# 1. Dependances et configuration locale
composer install

cp .env .env.local      # adapter DATABASE_URL / MAILER_DSN / APP_SECRET

# 2. Base de donnees
php bin/console doctrine:database:create
php bin/console doctrine:migrations:migrate

# 3. Donnees de demonstration (optionnel)
php bin/console doctrine:fixtures:load

# 4. Serveur de developpement
php -S 0.0.0.0:8000 -t public
```

Comptes de démonstration : **admin@melody.test** (administrateur), **user@melody.test** et **lea@melody.test** (utilisateurs), mot de passe **Password123!**.

8. Configuration

Variable	Description
APP_ENV / APP_DEBUG	Environnement (dev, prod, test) et mode debug
APP_SECRET	Clé de signature des sessions, jetons CSRF et cookies
APP_URL	URL publique de l'application
DATABASE_URL	DSN Doctrine (MySQL 8, charset utf8mb4)
MAILER_DSN	Transport email (SMTP en production, null://null en développement)
MAILER_FROM	Expéditeur des emails applicatifs
CONTACT_TO_EMAIL	Destinataire du formulaire de contact

9. Tests

La suite s'exécute sur une base SQLite jetable : aucun serveur MySQL n'est requis. 9 tests et 24 assertions, tous au vert.

```
php bin/phpunit                # suite complete
php bin/console lint:yaml config
php bin/console lint:twig templates
```

Test	Vérification
Page d'accueil publique	Rendu du hero et statut 200
Pages publiques	/login, /register et /contact accessibles
Redirection anonyme	/morceaux renvoie vers /login
Connexion valide	Accès au catalogue après authentification
Mot de passe erroné	Message d'erreur affiché
Création de morceau	Formulaire, persistance et propriétaire
Playlist d'autrui	Réponse 403 (protection IDOR)
Back-office	403 pour ROLE_USER, 200 pour ROLE_ADMIN
Formulaire de contact	Message persisté et confirmation affichée

10. Déploiement Docker (production)

La pile fournie contient PHP-FPM (Symfony), nginx et MySQL 8, avec un volume partagé pour les fichiers publics et les uploads.

```
# Variables de production (ne pas committer)
cat > .env.prod <<'EOF'
APP_SECRET=...
APP_URL=https://melody.example.com
MYSQL_DATABASE=melody
MYSQL_USER=melody
MYSQL_PASSWORD=...
MAILER_DSN=smtp://user:pass@smtp.example.com:587
EOF

# Demarrage (migrations appliquees au boot)
docker compose --env-file .env.prod up -d --build

# Verifications
docker compose logs -f app
curl -I http://localhost:8080
```

Points d'attention en production

- Placer l'application derrière un reverse proxy TLS (Traefik, Caddy, nginx, Cloudflare).
- Sauvegarder le volume MySQL et le volume des uploads.
- Configurer un vrai transport SMTP via MAILER_DSN.
- Restreindre /admin (VPN ou liste d'IP) si nécessaire.
- Surveiller var/log/prod.log (JSON sur stderr).

11. Interface et lecteur audio

Le front a été reconstruit sans framework CSS : design system natif, thème sombre, sprite SVG, pochettes générées et barre de lecture persistante sur toutes les pages.

Élément	Mise en œuvre
Design system	public/assets/css/app.css : tokens, grille fluide, 3 paliers responsive
Lecteur	File d'attente, aléatoire, répétition, volume, raccourcis clavier
Pochettes	8 dégradés déterministes et mosaïques 2x2 pour les playlists

Élément	Mise en œuvre
Assets	Sprite SVG inline, zéro CDN, zéro police distante
Audio de démo	12 pistes WAV générées par docs/tools/make_demo_audio.py

12. Refonte 2.0 — corrections majeures

Problème initial	Correction apportée
encore_entry_link_tags sans WebpackEncore : toutes les pages plantaient	Assets Bootstrap 5 auto-hébergés
Extension Twig non autoloadable (namespace et PSR-4)	Supprimée, responsive géré en CSS
Morceaux::setImageFile écrivait un champ inexistant	Corrigé
User::getPassword() pouvait retourner null	Signature ?string conforme à l'interface
Suppression de morceau en GET sans CSRF	POST, jeton CSRF et contrôle du propriétaire
Formulaire de création sans isValid()	Validation complète
Templates appelant des routes inexistantes	Routes unifiées morceaux.*
Page de contact vide	Page complète
Fixtures invalides (arguments, flush manquant)	Fixtures réécrites
Email de contact avec From = visiteur	MAILER_FROM + Reply-To
access_control commenté : tout était public	Matrice d'accès explicite
Deux authentificateurs dont un TODO	Un seul LoginAuthenticator
/user exposait rôles et mots de passe	/profil et back-office ROLE_ADMIN
Docker en PostgreSQL pour des migrations MySQL	Pile MySQL 8, nginx et PHP-FPM
Symfony 6.1 en fin de vie	Symfony 6.4 LTS
EasyAdmin installé mais inutilisé	Back-office complet
Uploads en collision avec la route /morceaux/{id}	Uploads sous /media/
Validation métier quasi absente	Contraintes Assert sur toutes les entités

13. Roadmap

- Playlists publiques ou privées avec lien de partage.
- Lecteur audio persistant (lecture continue).
- API REST / JSON pour un client mobile.
- Recommandations par genre et historique d'écoute.
- CI GitHub Actions : tests et build Docker.
- Traductions EN / ES.

14. Licence

MIT License — Copyright (c) 2025 Alex Alkhatib. Projet personnel réalisé pour approfondir Symfony, Doctrine ORM et la conception d'applications web sécurisées.

Document généré depuis le README du projet (docs/tools/build_pdf.py).